

AI 2002

Artificial Intelligence

Lecture 20

Mahzaib Younas

Lecturer, Department of Computer Science

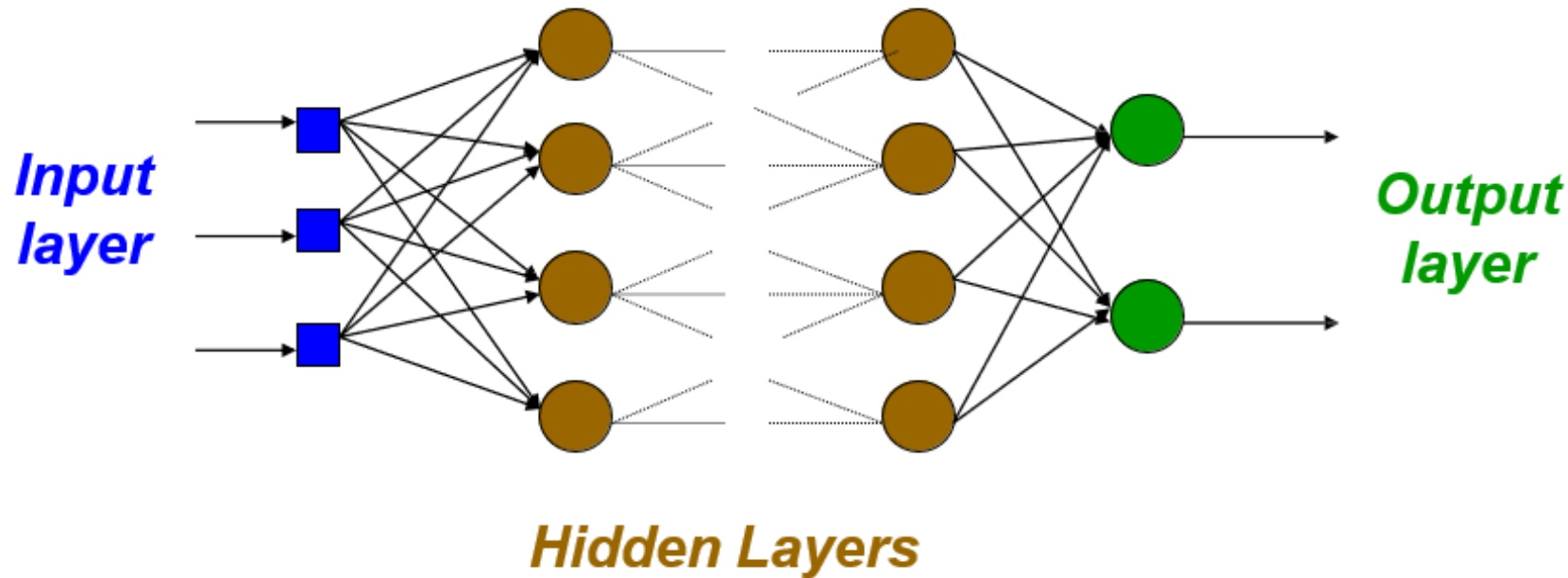
FAST NUCES CFD

Multi Layer Perceptron

- A class of **feedforward neural network**
- Consists of **input layer, hidden layer(s), and output layer**
- Uses **nonlinear activation functions**

Multi Layer Architecture Perceptron

- MLP used to describe any general feedforward (no recurrent connections) network



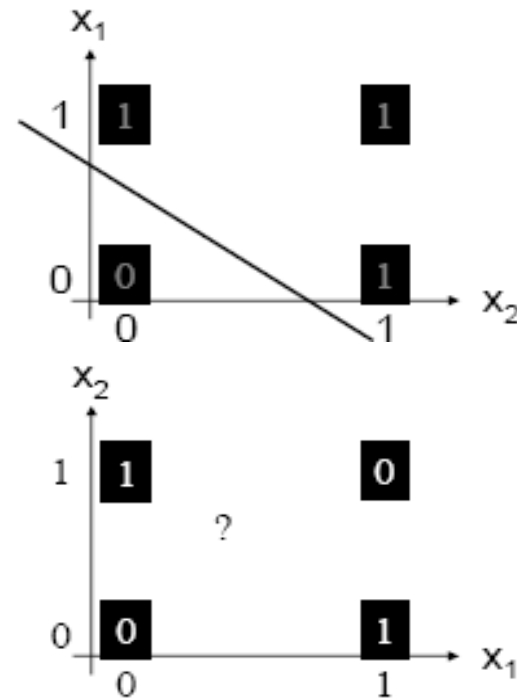
Multi Layer Network

OR function

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

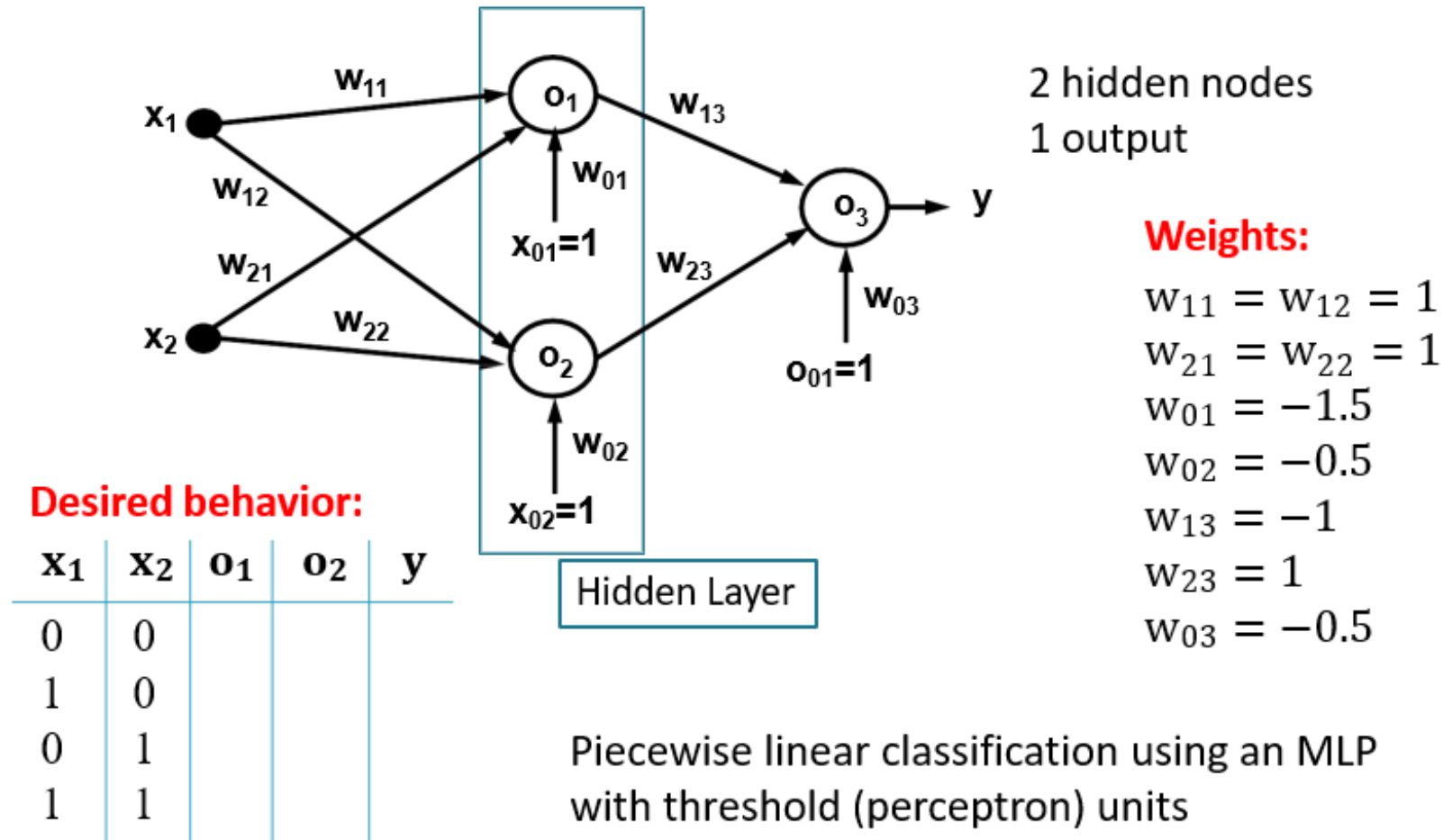
XOR function

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



Multi Layer Network

Threshold :
output < 0 0
Output > 0 1



Solution:

- **Inputs:** x_1, x_2
Hidden Layer Nodes: o_1, o_2
Output Node: $o_3 = y$
- Activation/Threshold Function

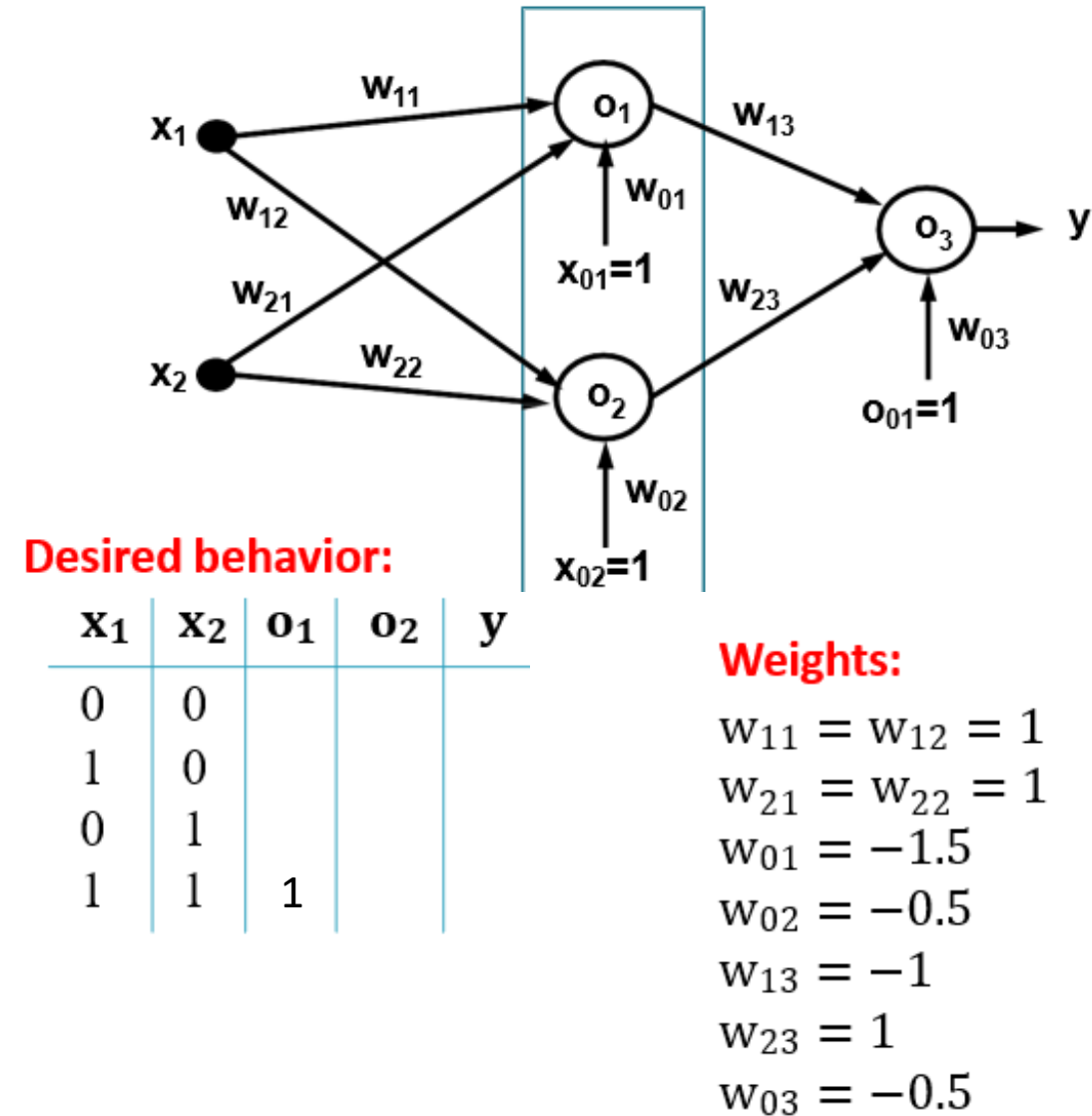
$$\text{Output} = \begin{cases} 1 & \text{if input sum} \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Solution

- If $x_1 = 1$ and $x_2 = 1$
- Calculate the output of hidden layers

$$O_1 = (x_{01}w_{01} + x_1w_{11} + x_2w_{21})$$
$$O_1 = (1 * -1.5) + (1 * 1) + (1 * 1)$$
$$= -1.5 + 1 + 1$$
$$= 0.5$$

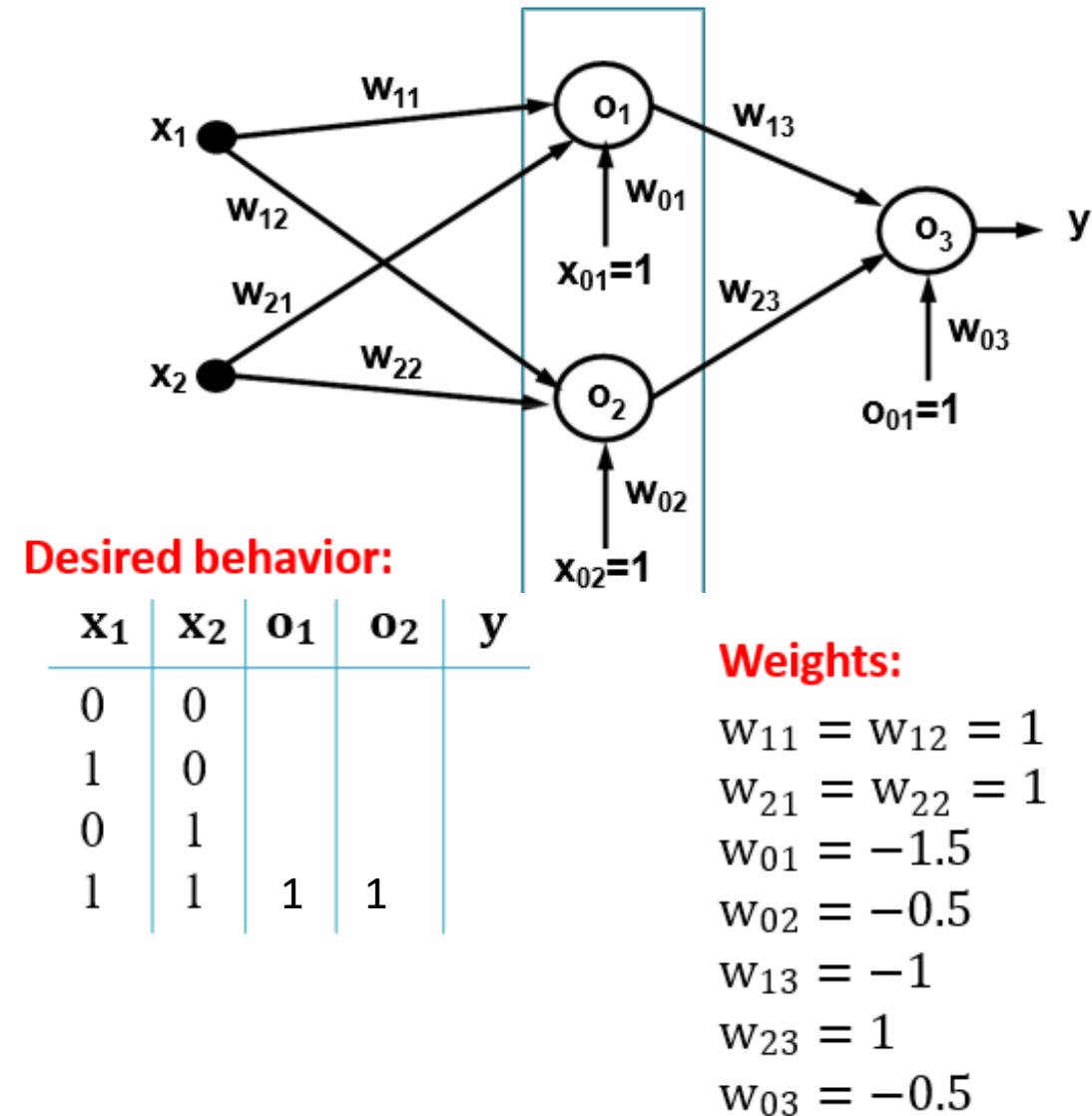
Output after threshold of $O_1 = 1$



Solution

- If $x_1 = 1$ and $x_2 = 1$
- Calculate the output of hidden layers
$$O_2 = (x_{02}w_{02} + x_1w_{12} + x_2w_{22})$$
$$O_2 = (1 * -0.5) + (1 * 1) + (1 * 1)$$
$$= -0.5 + 1 + 1$$
$$= 1.5$$

Output after threshold of $O_2 = 1$



Solution

- If $O_1 = 1$ and $O_2 = 1$
- Calculate the output of final layers

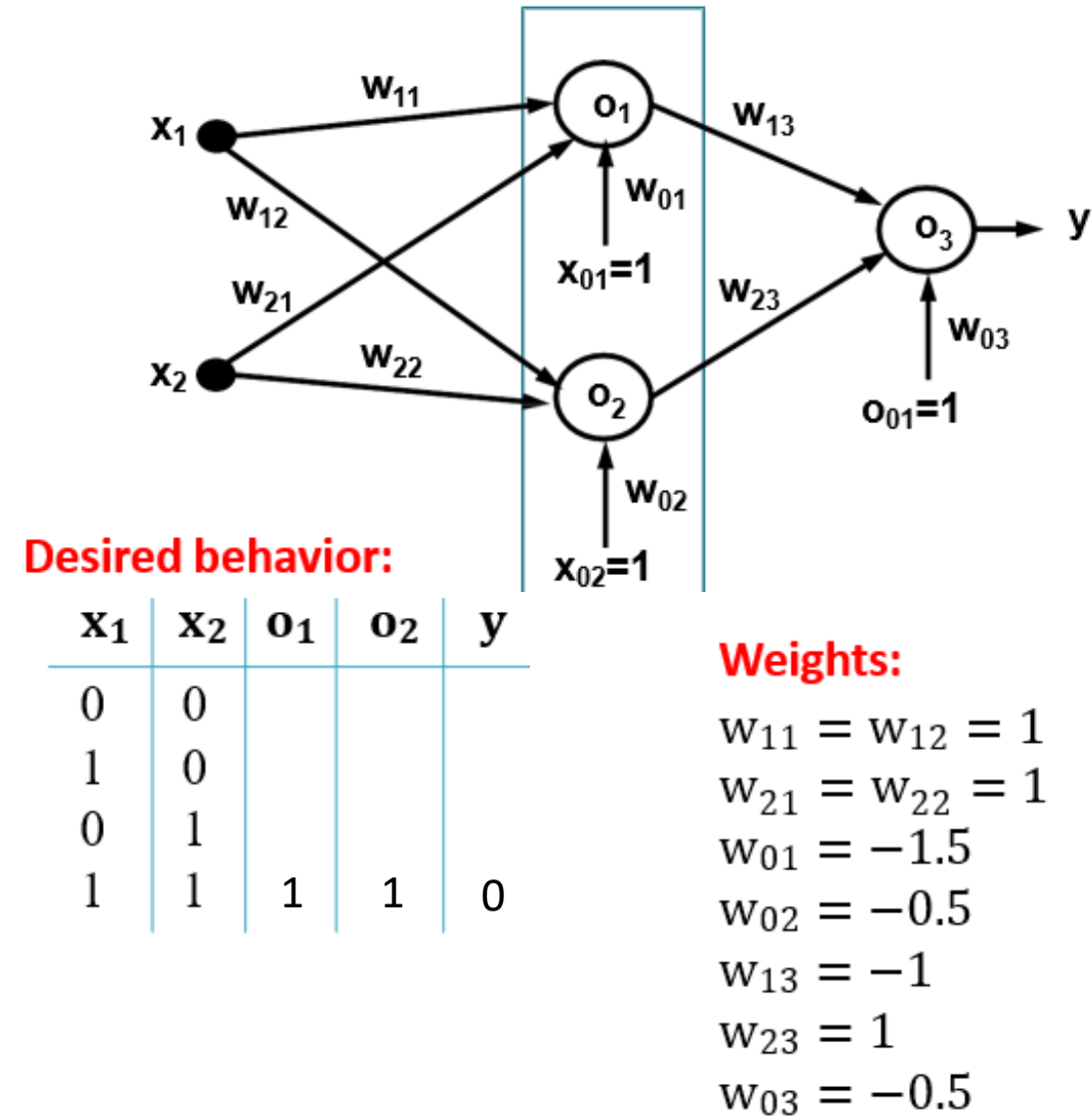
$$O_3 = (o_1 w_{13} + o_2 w_{23} + o_{01} w_{03})$$

$$O_2 = (1 * -1) + (1 * 1) + (1 * -0.5)$$

$$= -1 + 1 + (-0.5)$$

$$= -0.5$$

Output after threshold of $O_3 = 0$

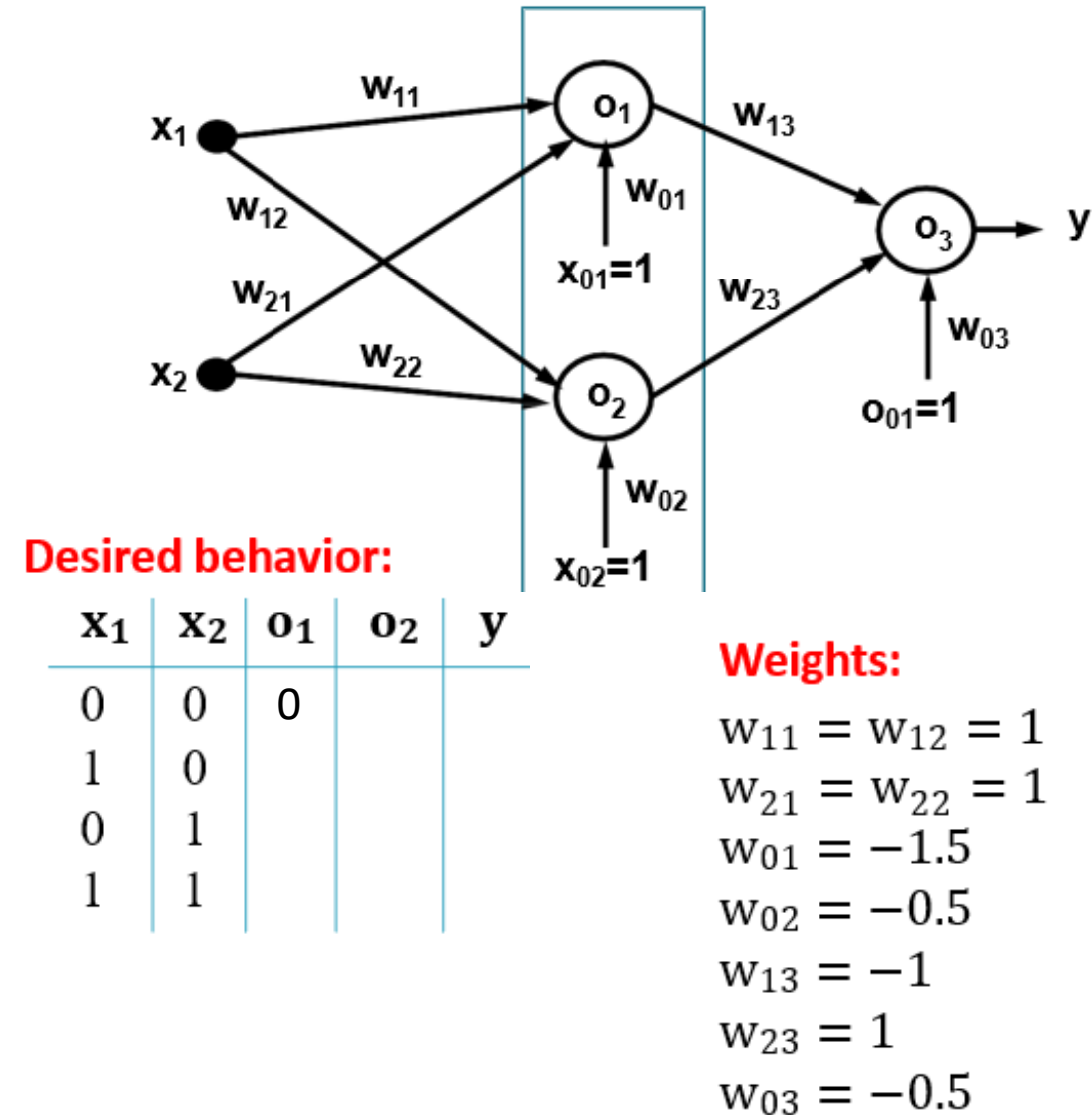


Solution

- If $x_1 = 0$ and $x_2 = 0$
- Calculate the output of hidden layers

$$\begin{aligned} O_1 &= (x_{01}w_{01} + x_1w_{11} + x_2w_{21}) \\ O_1 &= (1 * -1.5) + (0 * 1) + (0 * 1) \\ &= -1.5 + 0 + 0 \\ &= -1.5 \end{aligned}$$

Output after threshold of $O_1 = 0$

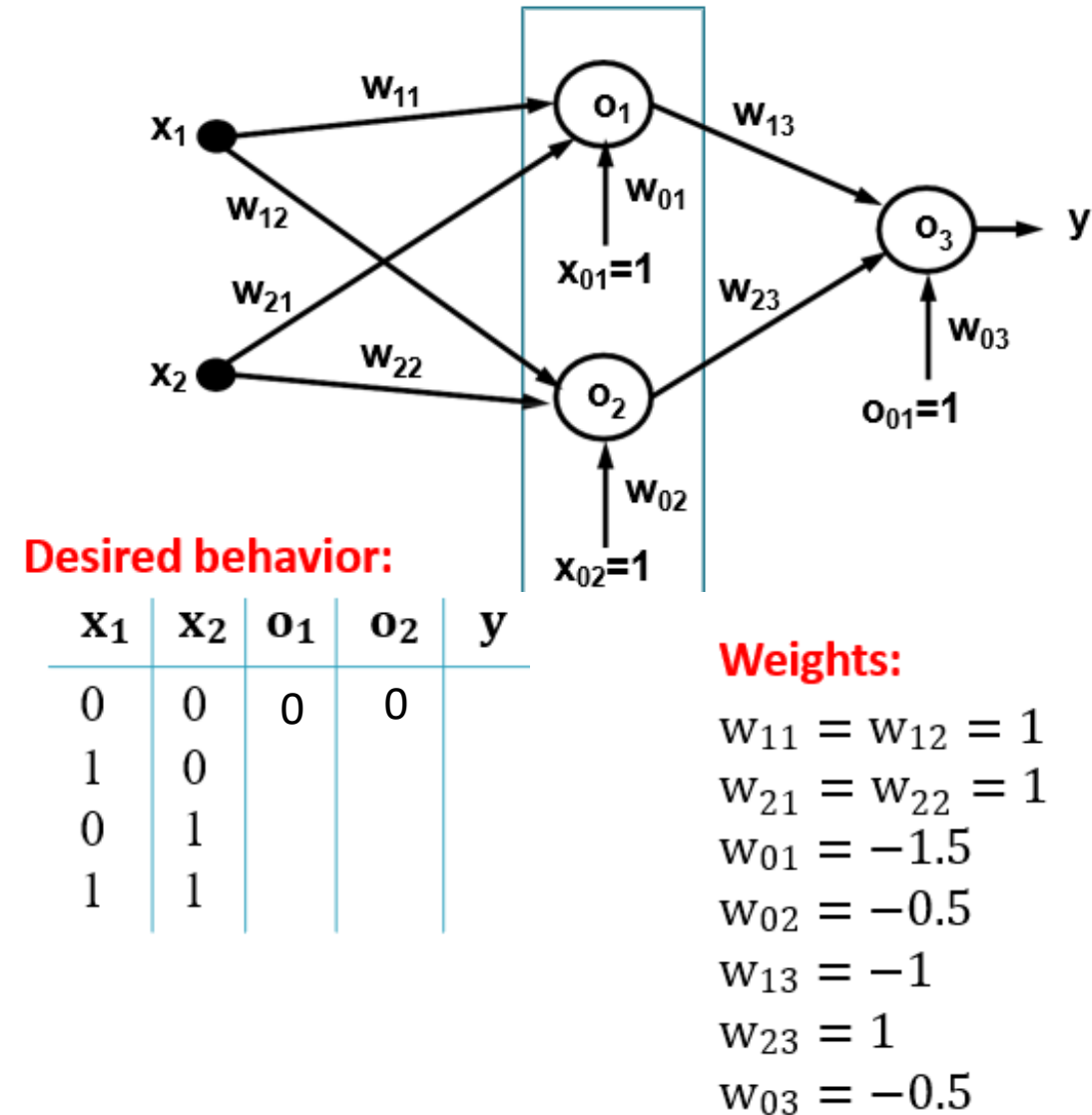


Solution

- If $x_1 = 0$ and $x_2 = 0$
- Calculate the output of hidden layers

$$\begin{aligned} O_2 &= (x_{02}w_{02} + x_1w_{12} + x_2w_{22}) \\ O_2 &= (1 * -0.5) + (0 * 1) + (0 * 1) \\ &= -0.5 + 0 + 0 \\ &= -0.5 \end{aligned}$$

Output after threshold of $O_2 = 0$



Solution

- If $O_1 = 0$ and $O_2 = 0$
- Calculate the output of final layers

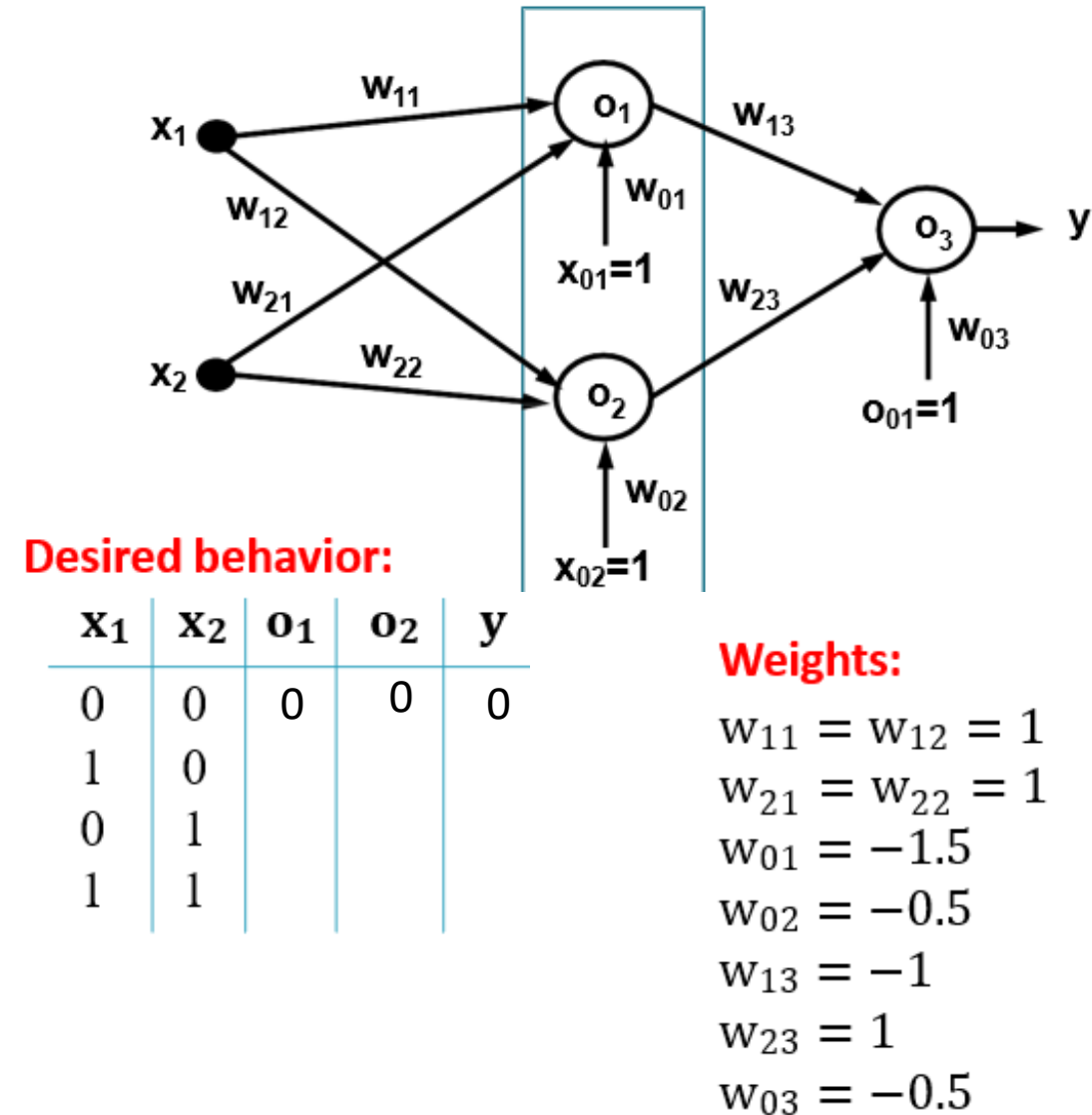
$$O_3 = (o_1 w_{13} + o_2 w_{23} + o_{01} w_{03})$$

$$O_2 = (0 * -1) + (0 * 1) + (1 * -0.5)$$

$$= -0 + 0 + (-0.5)$$

$$= -0.5$$

Output after threshold of $O_3 = 0$

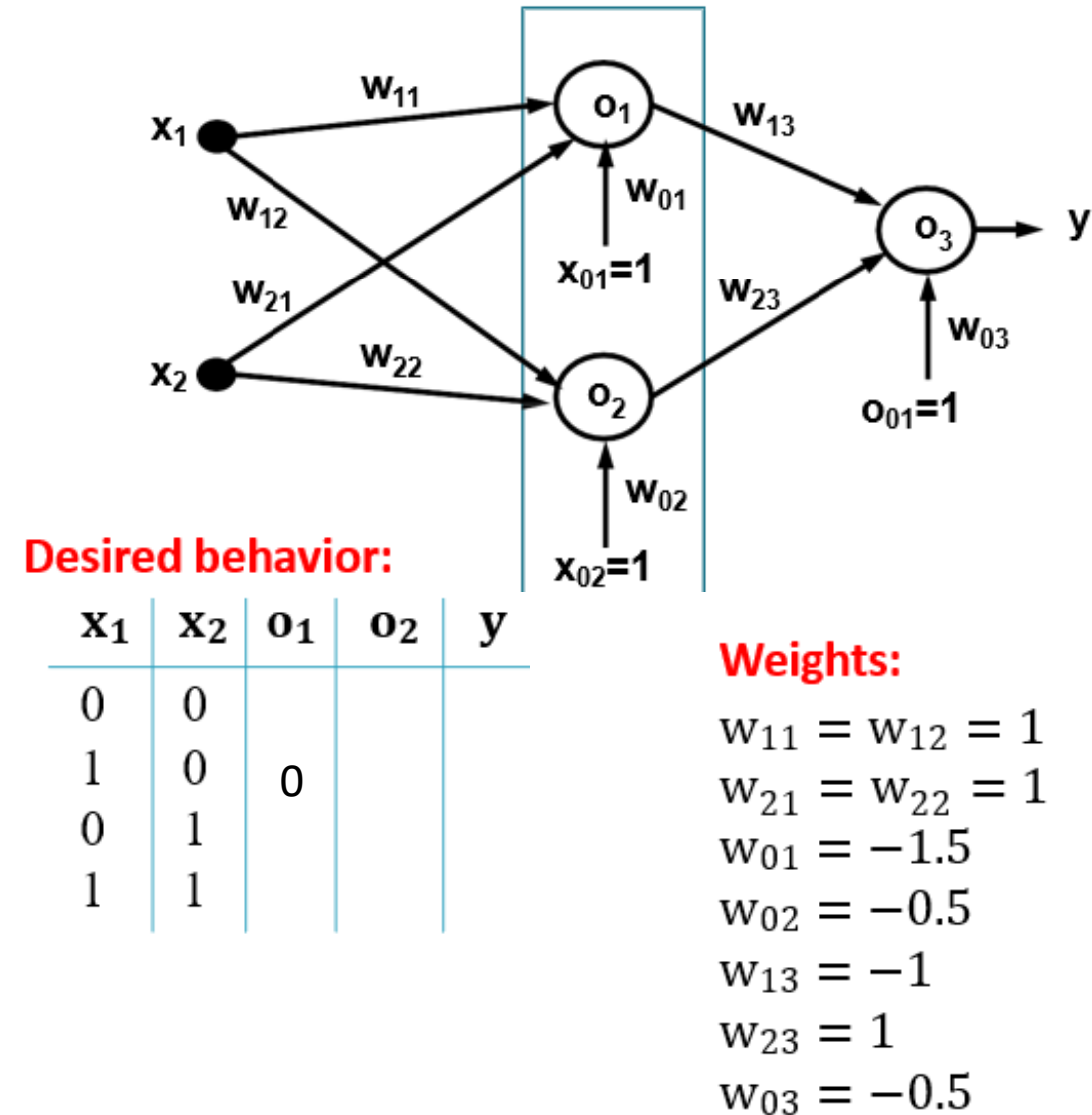


Solution

- If $x_1 = 1$ and $x_2 = 0$
- Calculate the output of hidden layers

$$\begin{aligned} O_1 &= (x_{01}w_{01} + x_1w_{11} + x_2w_{21}) \\ O_1 &= (1 * -1.5) + (1 * 1) + (0 * 1) \\ &= -1.5 + 1 + 0 \\ &= -0.5 \end{aligned}$$

Output after threshold of $O_1 = 0$



Solution

- If $x_1 = 1$ and $x_2 = 0$
- Calculate the output of hidden layers

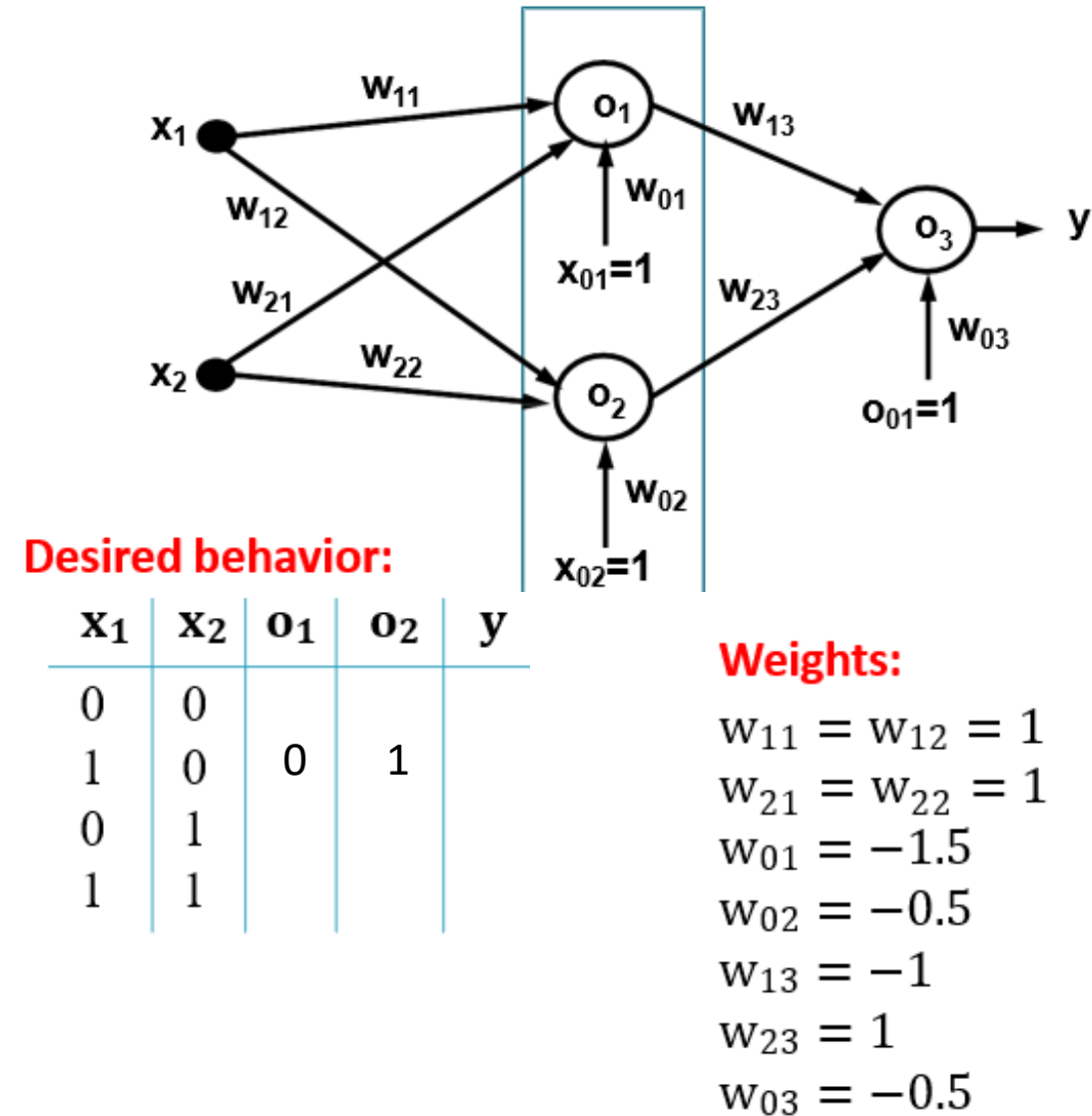
$$O_2 = (x_{02}w_{02} + x_1w_{12} + x_2w_{22})$$

$$O_2 = (1 * -0.5) + (1 * 1) + (0 * 1)$$

$$= -0.5 + 1 + 0$$

$$= 0.5$$

Output after threshold of $O_2 = 1$



Solution

- If $O_1 = 0$ and $O_2 = 1$
- Calculate the output of final layers

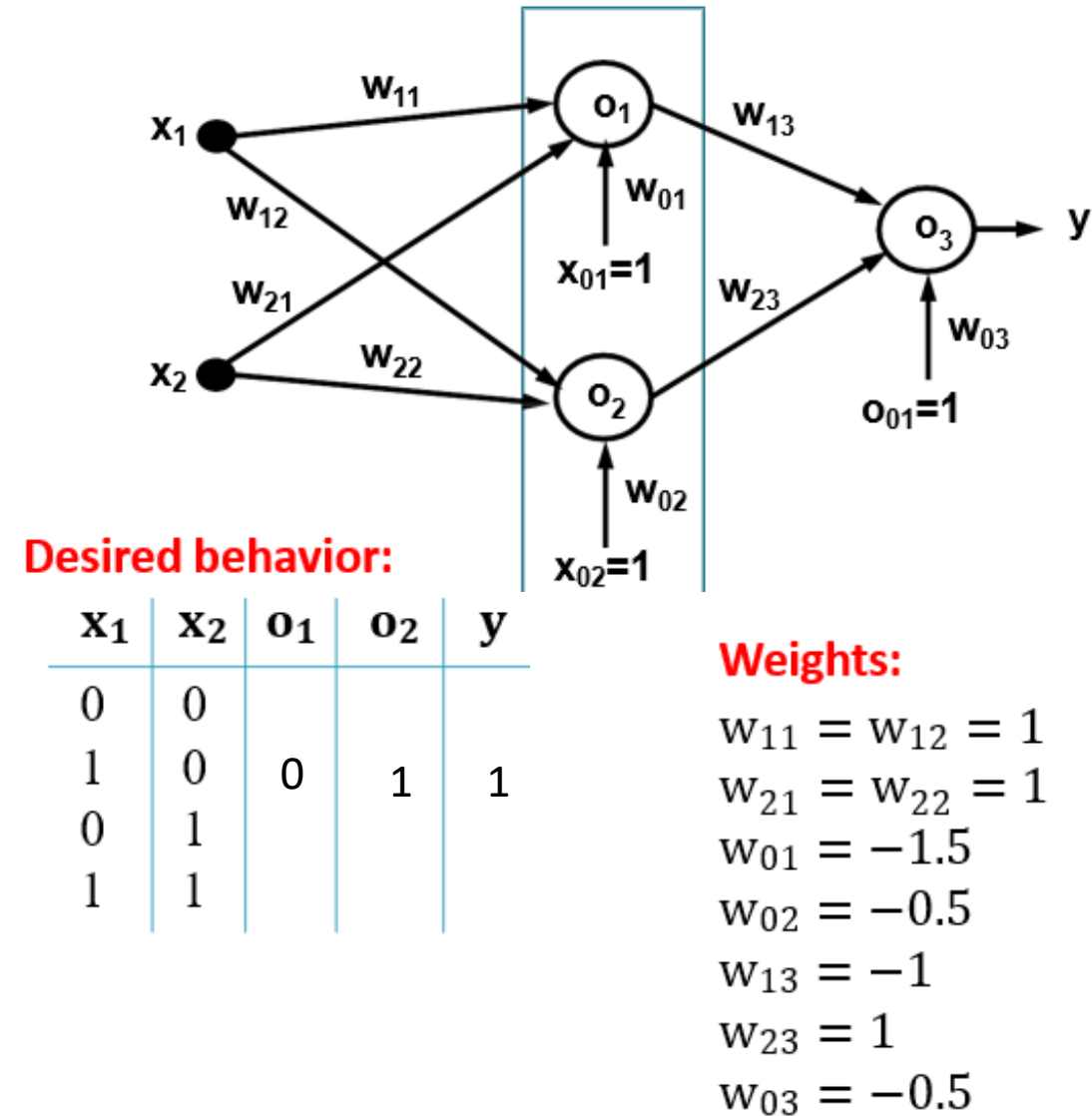
$$O_3 = (o_1 w_{13} + o_2 w_{23} + o_{01} w_{03})$$

$$O_2 = (0 * -1) + (1 * 1) + (1 * -0.5)$$

$$= -0 + 1 + (-0.5)$$

$$= 0.5$$

Output after threshold of $O_3 = 1$

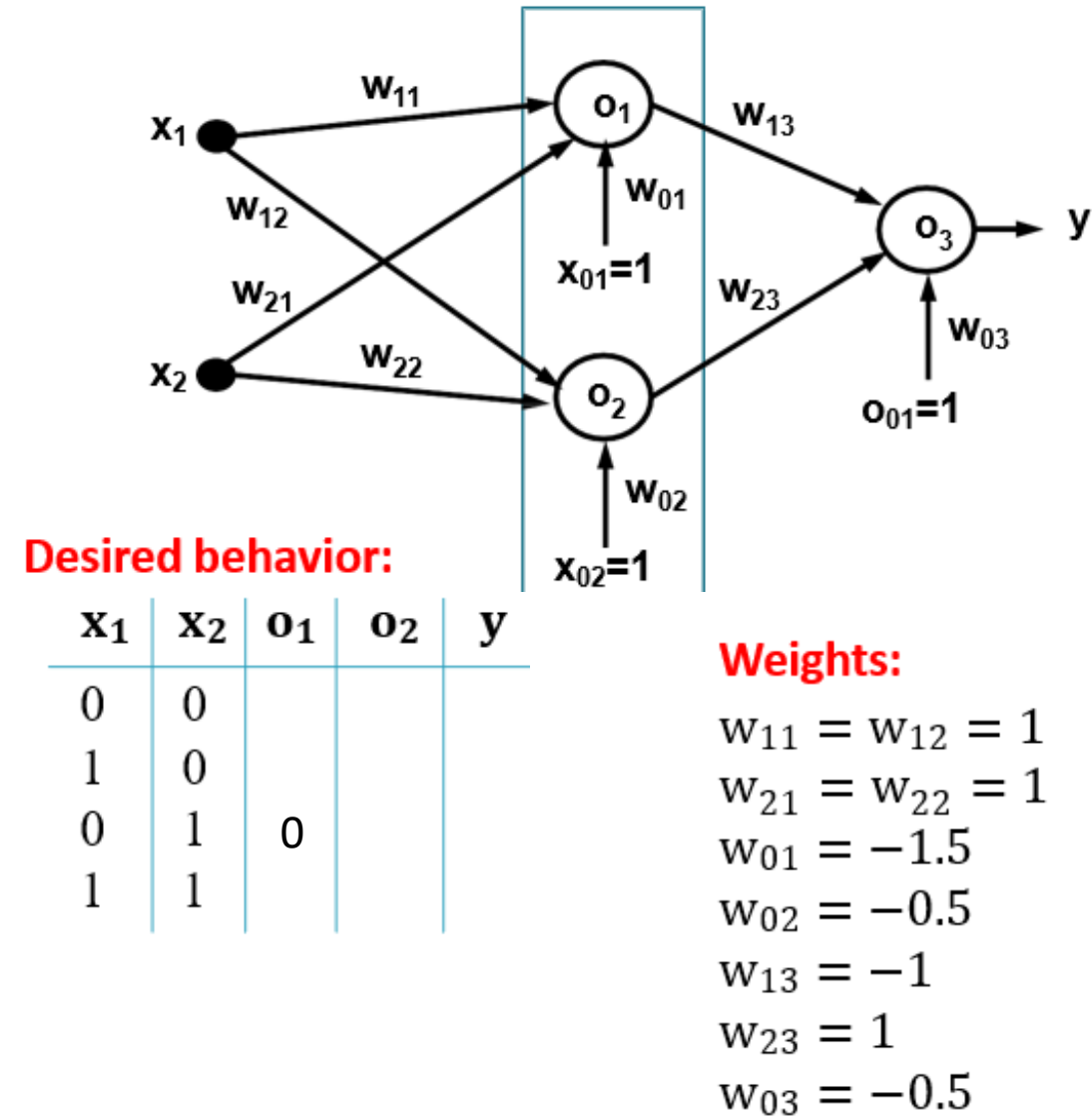


Solution

- If $x_1 = 0$ and $x_2 = 1$
- Calculate the output of hidden layers

$$\begin{aligned} O_1 &= (x_{01}w_{01} + x_1w_{11} + x_2w_{21}) \\ O_1 &= (1 * -1.5) + (0 * 1) + (1 * 1) \\ &= -1.5 + 0 + 1 \\ &= -0.5 \end{aligned}$$

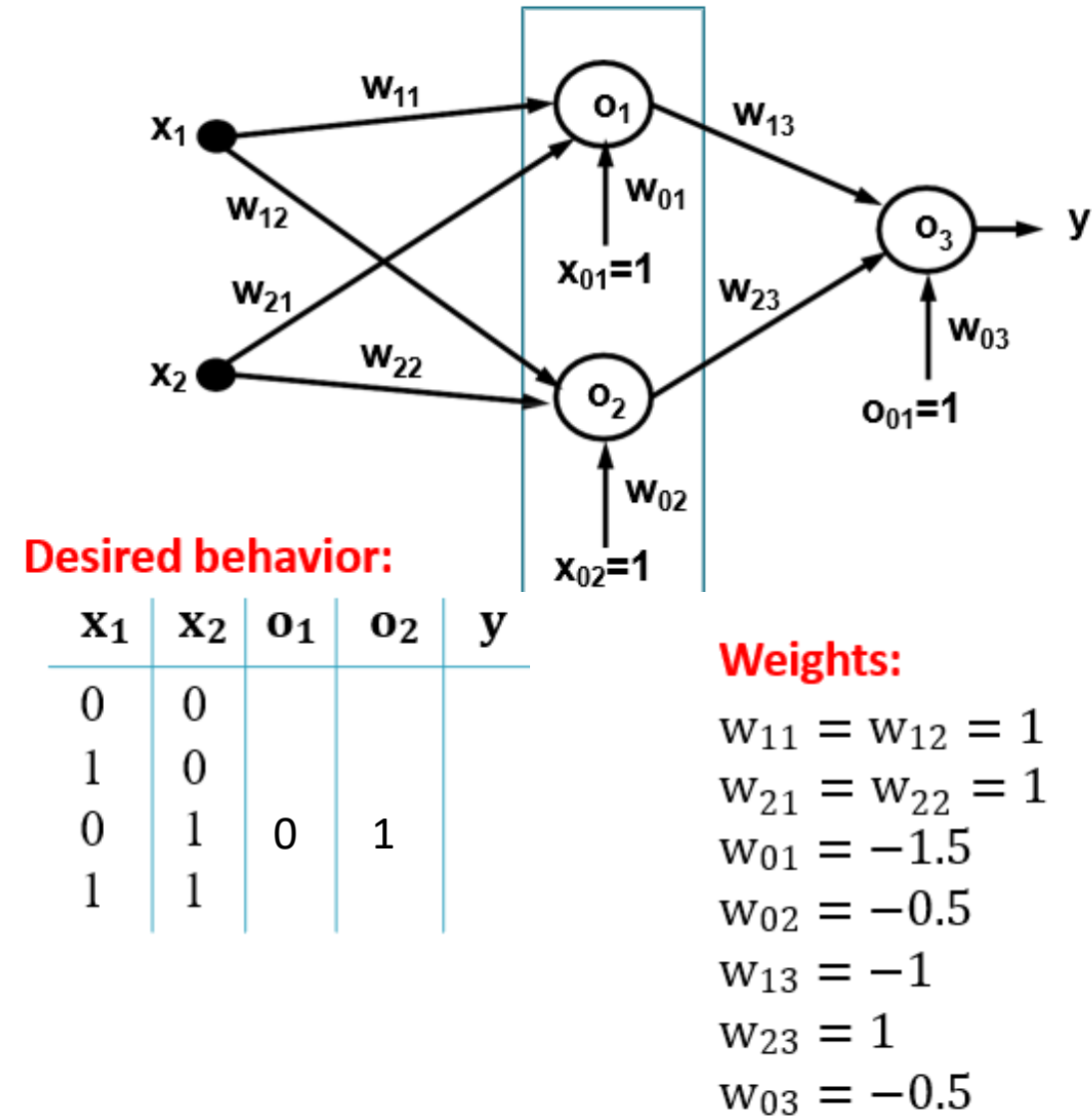
Output after threshold of $O_1 = 1$



Solution

- If $x_1 = 0$ and $x_2 = 1$
- Calculate the output of hidden layers
$$O_2 = (x_{02}w_{02} + x_1w_{12} + x_2w_{22})$$
$$O_2 = (1 * -0.5) + (0 * 1) + (1 * 1)$$
$$= -0.5 + 0 + 1$$
$$= 0.5$$

Output after threshold of $O_2 = 1$



Solution

- If $O_1 = 0$ and $O_2 = 1$
- Calculate the output of final layers

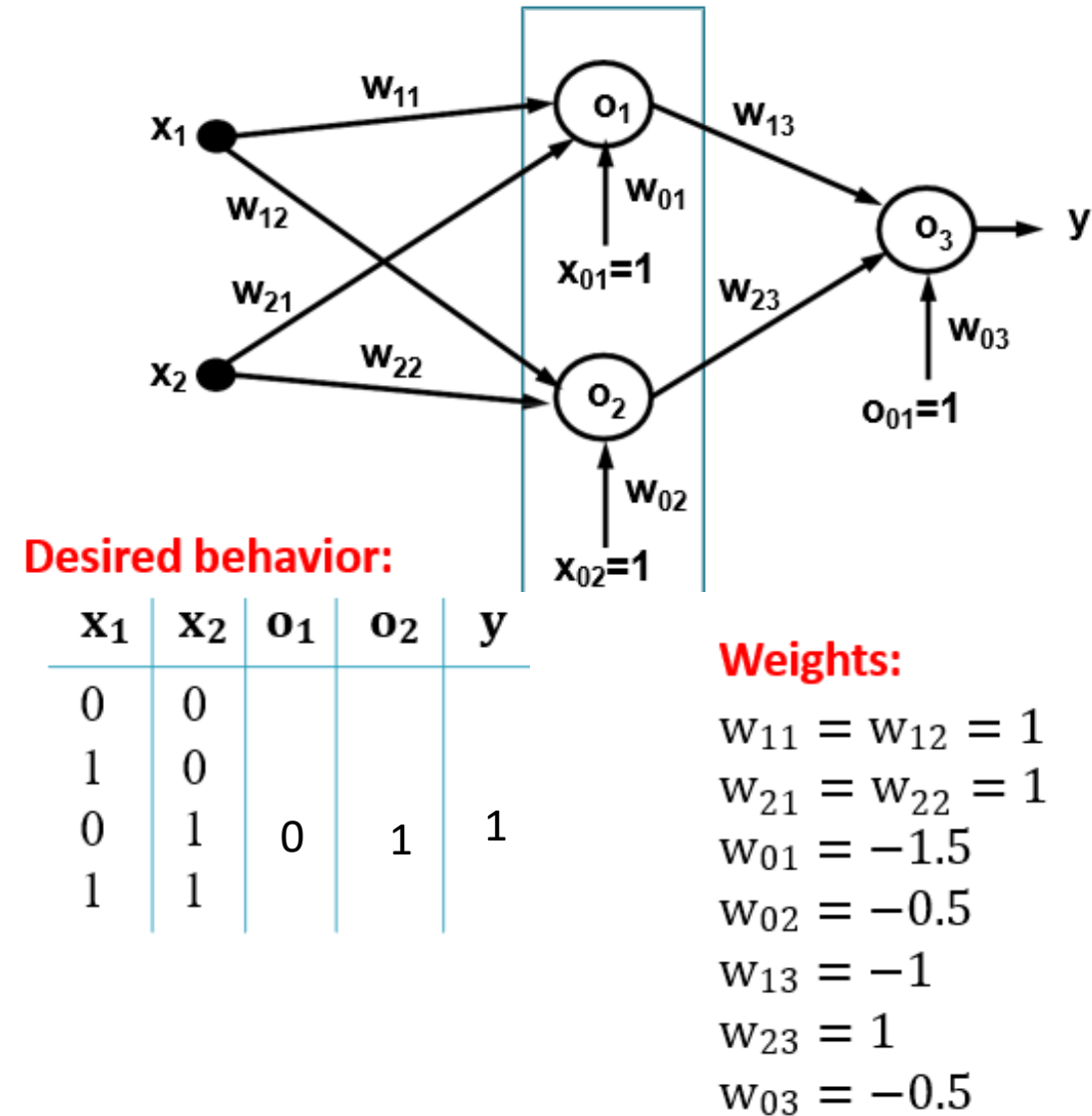
$$O_3 = (o_1 w_{13} + o_2 w_{23} + o_{01} w_{03})$$

$$O_2 = (0 * -1) + (1 * 1) + (1 * -0.5)$$

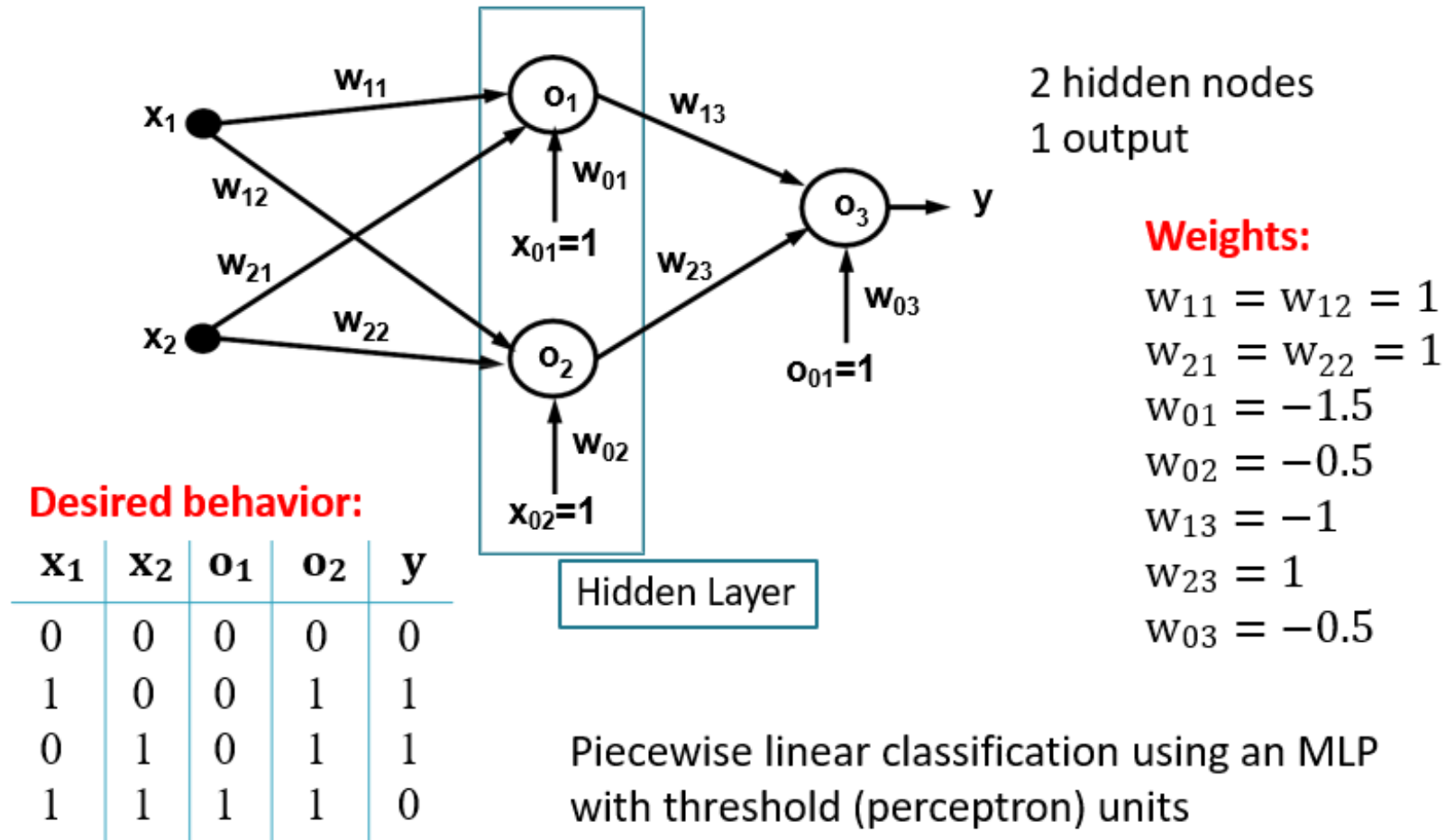
$$= -0 + 1 + (-0.5)$$

$$= 0.5$$

Output after threshold of $O_3 = 1$



Multi Layer Network



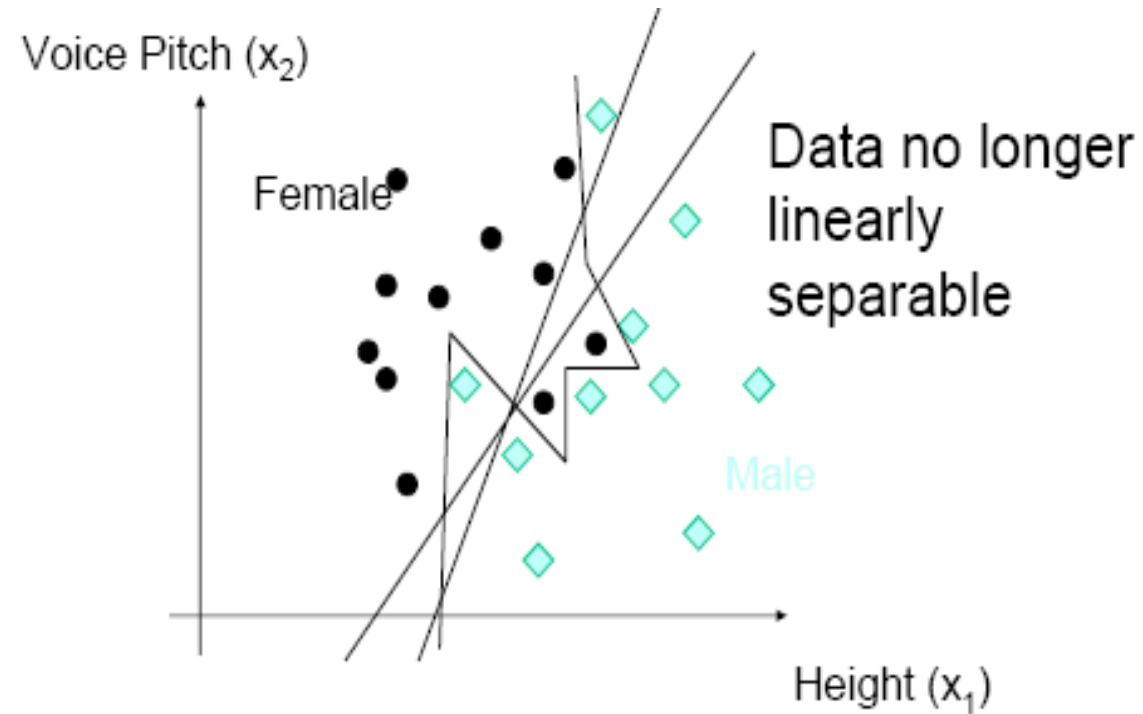
Multi Layer Network

- The single perceptron can only express **linear decision surfaces**.
- The kind of **multilayer networks** learned by the **back propagation** algorithm are capable of expressing a rich variety of **nonlinear decision surfaces**.

Multi Layer Network

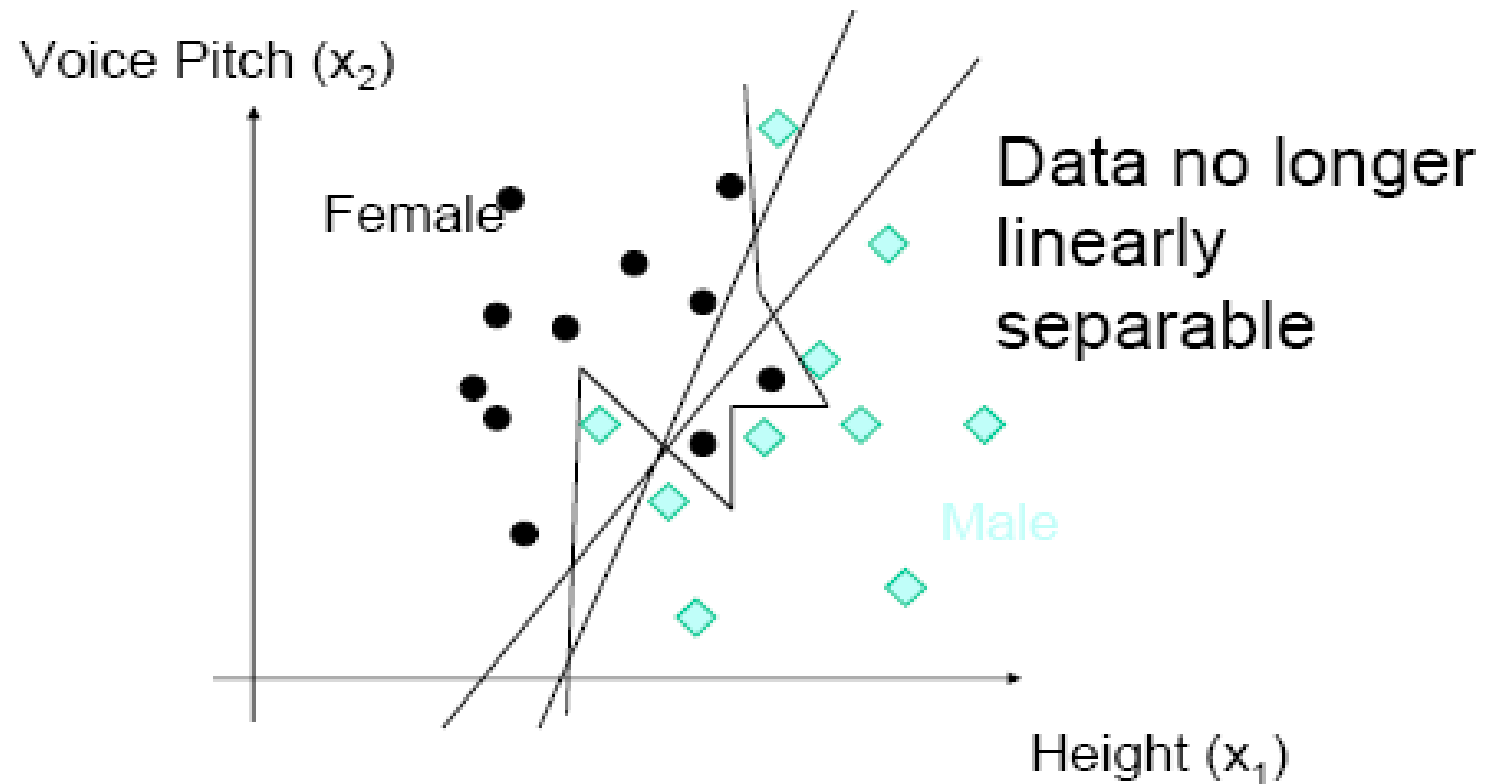
- The single perceptron can only express **linear decision surfaces**.
- The kind of **multilayer networks** learned by the **back propagation** algorithm are capable of expressing a rich variety of **nonlinear decision surfaces**.

Multi Layer Network



What is a good decision boundary ?

Multilayer Networks... Example



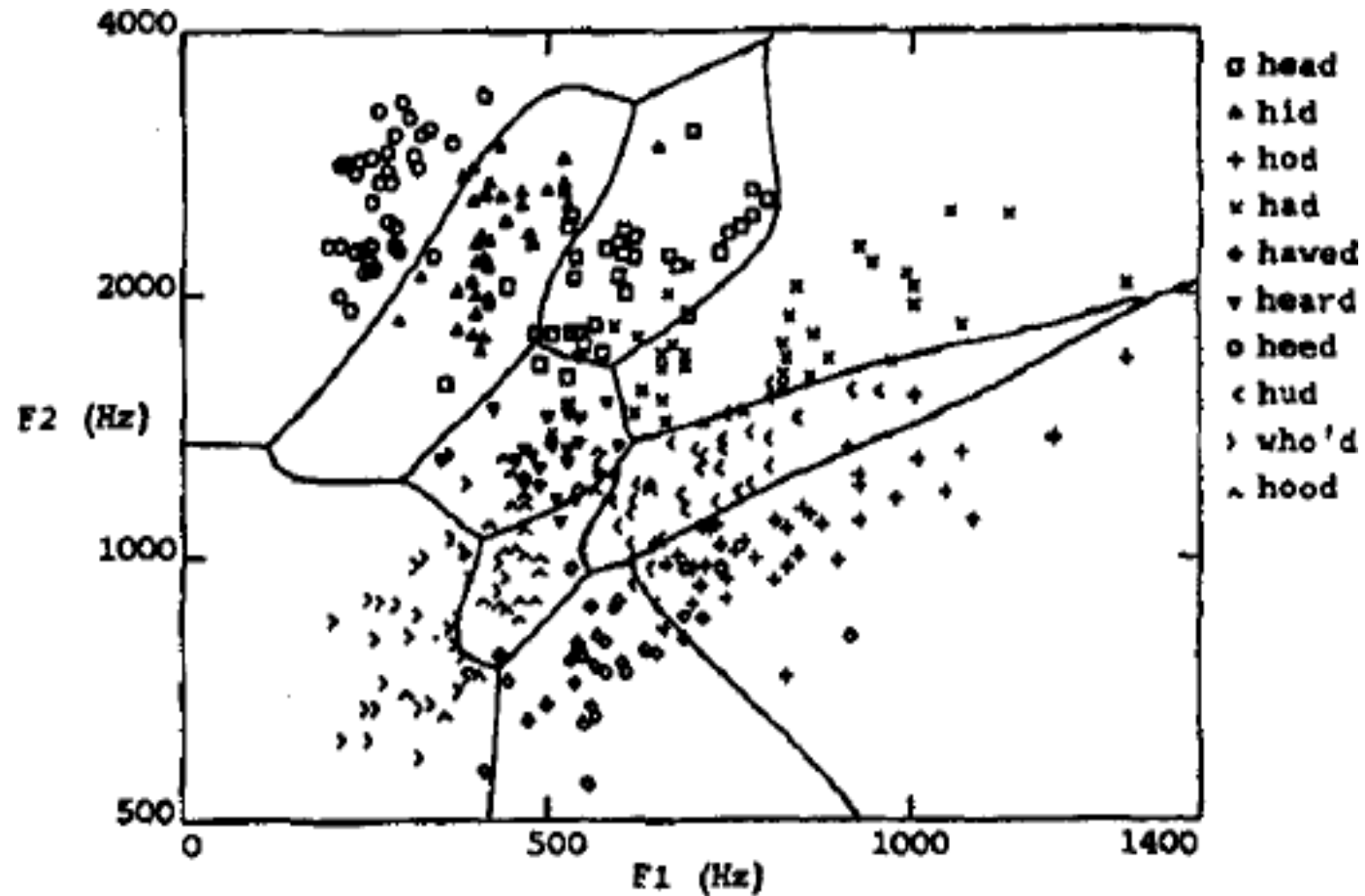
What is a good decision boundary ?

Multi Layer Network

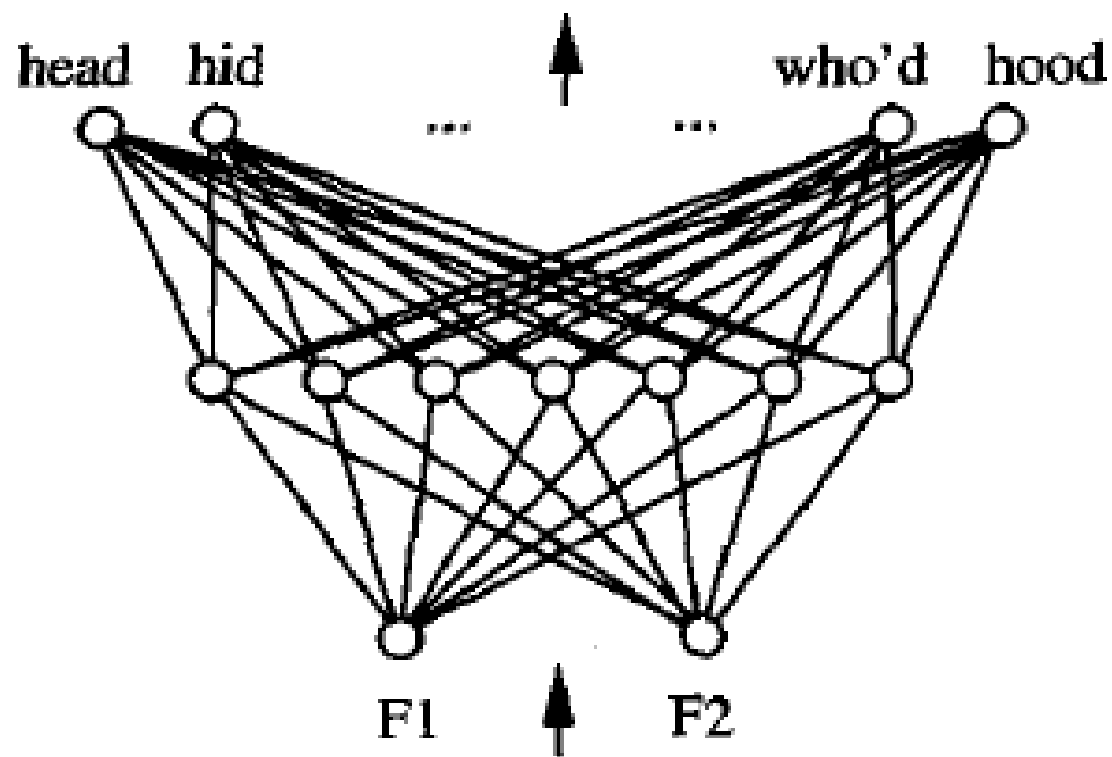
- **Example:**

- The speech recognition task involves distinguishing among 10 possible vowels, all spoken in the context of "h-d" (i.e., "hid," "had," "head," "hood," etc.).

Multi Layer Network



Multi Layer Network



Multi Layer Network

- What type of unit shall we use as the basis for constructing multilayer networks?
- Multiple layers of cascaded linear units still produce only linear functions, and we prefer networks capable of representing highly nonlinear functions
- The perceptron unit is another possible choice, is it?
 - its discontinuous threshold makes it undifferentiable and
 - hence unsuitable for gradient descent.

Solution:

- One solution is the **sigmoid unit:**
 - a unit very much like a perceptron, but based on a
 - **smoothed, differentiable threshold function.**
- Like the **perceptron**, the sigmoid unit
 - first computes a linear combination of its inputs,
 - then applies a threshold to the result.

Sigmoid Unit

- In case of **sigmoid unit**, however, the **threshold output is a continuous function** of its input.
- More precisely, the sigmoid unit computes its output o as,

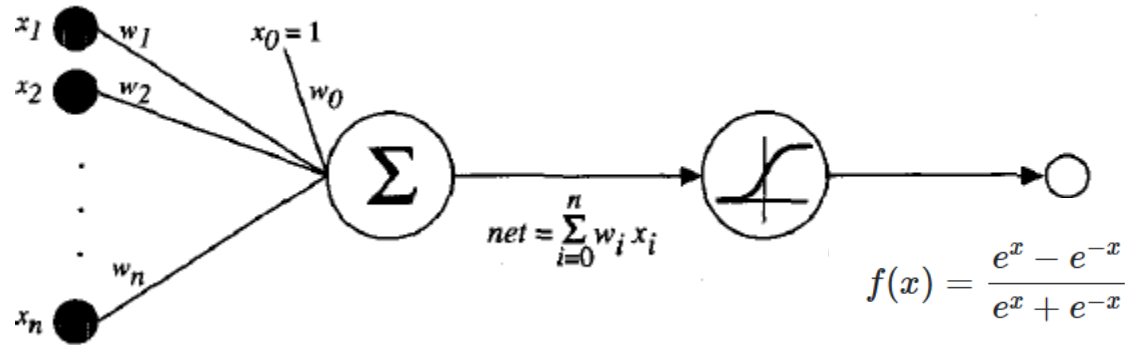
$$o = \sigma(\vec{w} \cdot \vec{x})$$

$$\sigma(y) = \frac{1}{1 + e^{-y}}$$

- σ is often called the sigmoid function or, alternatively, the logistic function.

The **sigmoid function** is a commonly used activation function in neural networks, especially in early architectures. It maps any real-valued number into the range between 0 and 1.

Tanh Function



Inputs ($x_1, x_2, \dots, x_n$)

Weights ($w_1, w_2, w_3, \dots, w_n$)

Weighted Sum (net):

$$net = \sum_{i=0}^n w_i x_i$$

Tanh Activation Function:

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

The sigmoid function squashes the output between -1 and 1. This value o becomes the neuron's output.

Example:

- Suppose we have given input features values
 - $x=[1,0,1,1]$
- We have four target output classes with weights

Class Name	Weights[w_0, w_1, w_2, w_3, w_4]
Good	$[-1.2, 0.8, 0.5, -0.6, 1.0]$
Bad	$[0.5, -1.0, 1.2, 1.5, -0.8]$
Average	$[-0.7, 1.5, -0.9, 1.2, 0.3]$
Excellent	$[1.0, -0.5, 0.7, -1.0, 0.9]$

Step 1: Apply Sigmoid Function

$$\text{net} = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4$$

$$\text{output} = \sigma(\text{net}) = \frac{1}{1 + e^{-\text{net}}}$$

Step 2=calculate for each class

- Class Good:

$$= -1.2 + (0.8)(1) + (0.5)(0) + (-0.6)(1) + (1.0)(1) = -1.2 + 0.8 + 0 - 0.6 + 1 = 0.0$$

$$\sigma(0.0) = \frac{1}{1 + e^0} = 0.5$$

- Class Bad:

$$= 0.5 + (-1.0)(1) + (1.2)(0) + (1.5)(1) + (-0.8)(1) = 0.5 - 1.0 + 0 + 1.5 - 0.8 = 0.2$$

$$\sigma(0.2) = \frac{1}{1 + e^{-0.2}} \approx 0.5498$$

Step 2=calculate for each class

- Class Average:

$$= -0.7 + (1.5)(1) + (-0.9)(0) + (1.2)(1) + (0.3)(1) = -0.7 + 1.5 + 0 + 1.2 + 0.3 = 2.3$$

$$\sigma(2.3) = \frac{1}{1 + e^{-2.3}} \approx 0.9089$$

- Class Excellent:

$$= 1.0 + (-0.5)(1) + (0.7)(0) + (-1.0)(1) + (0.9)(1) = 1.0 - 0.5 + 0 - 1.0 + 0.9 = 0.4$$

$$\sigma(0.4) = \frac{1}{1 + e^{-0.4}} \approx 0.5987$$

Final Sigmoid Output

Class Name	Sigmoid Function Values
Good	0.5
Bad	0.5498
Average	0.9089
Excellent	0.5987

Sigmoid Function

- Sigmoid function maps a very large input domain to a small range of outputs, it is often referred to as the *squashing function* of the unit.
- The sigmoid function has the useful property that **its derivative is easily expressed in terms of its output.**

$$\sigma(y) = \frac{1}{1 + e^{-y}}$$

$$\frac{d\sigma(y)}{dy} = \sigma(y) \cdot (1 - \sigma(y))$$

